

Text Vectorization:

Machine learning models cannot work with raw text directly; text must first be converted into a numerical representation before it can be used for training. This process is called vectorization. This article covers the main approaches used to convert text into numbers: Bag of Words, TF-IDF, and word embedding's along with the key differences between traditional vectorization methods and embedding's.

Bag of Words

Bag of Words is one of the simplest techniques used to convert text into numerical form. It works by counting how many times each word appears in a document, without considering grammar, word order, or context. Every unique word across the entire dataset becomes a separate column (feature) and each document is represented as a row containing the count of each word.

Code and Screenshot:

```

  ✓ 1s [1] from sklearn.feature_extraction.text import CountVectorizer

      sample_reviews = [
          "this movie was great and the acting was great",
          "this movie was boring and too long",
          "great acting, terrible story"
      ]

      vectorizer = CountVectorizer()
      bow_matrix = vectorizer.fit_transform(sample_reviews)

      print(vectorizer.get_feature_names_out())
      print(bow_matrix.toarray())

  ... ['acting' 'and' 'boring' 'great' 'long' 'movie' 'story' 'terrible' 'the'
       'this' 'too' 'was']
      [[1 1 0 2 0 1 0 0 1 1 0 2]
       [0 1 1 0 1 1 0 0 0 1 1 1]
       [1 0 0 1 0 0 1 1 0 0 0 0]]
```

In the first sentence, the word "great" appears twice, which is why its count is 2 in the resulting vector. This is the core idea behind Bag of Words: each document is reduced to a simple count of how often each word occurs.

TF-IDF

TF-IDF (Term Frequency–Inverse Document Frequency) addresses the limitation of Bag of Words by weighting words according to how important they actually are, rather than relying purely on raw frequency. A word that occurs frequently within a specific document but rarely across the rest of the dataset is assigned a higher score, while a word that occurs frequently across most documents is assigned a lower score, even without explicitly removing stop words.

TF-IDF is composed of two components:

- **Term Frequency (TF)** measures how often a word appears within a given document.

- **Inverse Document Frequency (IDF)** measures how rare or common a word is across the entire dataset. Common words receive a low IDF value, while rare or distinctive words receive a high IDF value.

The final TF-IDF score for a word is calculated by multiplying these two values resulting in a score that favours words that are frequent in a specific document but not widespread across the dataset.

Code and Screenshot:

TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(sample_reviews)

print(tfidf_vectorizer.get_feature_names_out())
print(tfidf_matrix.toarray().round(2))

... ['acting' 'and' 'boring' 'great' 'long' 'movie' 'story' 'terrible' 'the'
      'this' 'too' 'was']
[[0.27 0.27 0.    0.54 0.    0.27 0.    0.    0.35 0.27 0.    0.54]
 [0.    0.33 0.43 0.    0.43 0.33 0.    0.    0.33 0.43 0.33]
 [0.43 0.    0.    0.43 0.    0.    0.56 0.56 0.    0.    0.    0. ]]
```

Compared to the Bag of Words output, the word "great" still carries a high value in the first row, but common words such as "was" and "and" are weighted down since they appear across multiple documents and are less distinctive. For this reason, TF-IDF is generally considered a stronger baseline than plain Bag of Words for most text classification tasks.

Introduction to Embedding's:

Both Bag of Words and TF-IDF share a common limitation: they treat words as isolated symbols with no relationship to one another. To a TF-IDF vectorizer, words such as "good" and "great" are simply two unrelated columns, with no indication that they carry similar meaning.

Word embedding's address this limitation by representing each word as a dense numerical vector (typically 100–300 dimensions), learned from patterns of word usage across large amounts of text. Words that appear in similar contexts are assigned similar vectors, allowing the model to capture semantic relationships between words rather than just their presence or frequency.

Code and Screenshot:

Introduction to Embeddings

```
[4] 1 pip install gensim

... Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
  Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
  Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
  Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.6.1)
  Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.2.2)
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
  27.9/27.9 MB 69.8 MB/s eta 0:00:00

Installing collected packages: gensim
Successfully installed gensim-4.4.0

[5] 1 import gensim.downloader as api

word_vectors = api.load("glove-wiki-gigaword-100")

print(word_vectors.most_similar("movie", topn=5))

... [('film', 0.9055121541023254), ('movies', 0.8959327340126038), ('films', 0.86635299949646), ('hollywood', 0.8239826560020447), ('comedy', 0.8141382932662964)]
```

Difference Between TF-IDF and Embedding's:

	TF-IDF	Embeddings
What it captures	Importance/rarity of a word within a specific dataset	Semantic meaning and relationships between words, learned from large external text corpora
Vector size	Equal to vocabulary size (often thousands of dimensions) mostly sparse	Fixed and relatively small (e.g., 100–300 dimensions), dense
Captures synonyms	No	Yes
Requires external training data	No, uses only the given dataset	Typically pretrained on large external corpora
Best suited for	Simple, interpretable baselines and smaller datasets	Tasks where context and meaning are important

References:

<https://www.geeksforgeeks.org/machine-learning/understanding-tf-idf-term-frequency-inverse-document-frequency/>

<https://www.geeksforgeeks.org/nlp/bag-of-words-bow-model-in-nlp/>

<https://www.geeksforgeeks.org/nlp/what-are-embedding-models/>